

Documento de Arquitectura

Sistema de Gestion de Ordenes Operativas (Sysman)

Robert Michael Rosero Salazar <https://robertrosero.com>

Este documento resume la arquitectura que use para el sistema de gestion de ordenes operativas: Spring Boot 3 / Java 21 con arquitectura hexagonal, Oracle como base de datos (con el procedimiento PL/SQL para la transicion de estado) y un frontend en Next.js.

Ambiente desplegado

- **Backend** <https://backend-production-89925.up.railway.app>
- **Swagger / OpenAPI** : <https://backend-production-89925.up.railway.app/swagger-ui/index.html>
- **Frontend (Vercel)**: <https://sysman.vercel.app/>
- **Base de datos**: Oracle Autonomous Database, nivel Always Free de Oracle Cloud Infrastructure (OCI).
- **Repositorio**: <https://github.com/bukohub/sysman>

Para probar la aplicación. Se puede entrar en <https://sysman.vercel.app> y revisar su funcionamiento. Desplugué el backend en Railway, el frontend fue desplegado en Vercel, y la base de datos en Oracle Cloud.

1. Arquitectura propuesta

Arquitectura hexagonal con 5 módulos Maven: sysman-domain (modelo puro, sin dependencias de framework), sysman-application (casos de uso y limite transaccional), sysman-infrastructure-persistence (JPA, Flyway, PL/SQL), sysman-infrastructure-rest (controladores v1, DTOs, manejo de errores) y sysman-bootstrap (ensamblaje y configuración por perfil). Se puede revisar la documentación API en esta enlace: <https://backend-production-89925.up.railway.app/swagger-ui/index.html>

Ejemplo de flujo (PUT /api/v1/orden/{id}/estado): el controlador valida el DTO y lo mapea a un comando; el servicio de aplicación (transaccional) valida la transición y luego invoca el adaptador JDBC que ejecuta el procedimiento PL/SQL en Oracle, que bloquea la fila, valida la transición y deja el registro en el histórico. El frontend (React con Next.js) consume la API con Server Components para listado/detalle y Client Components para los formularios.

2. Justificación técnica

Hexagonal multi-modulo: para un sistema con reglas de negocio que deben sobrevivir cambios de tecnología (BD, canal de entrada) y que exige alta confiabilidad transaccional, la arquitectura hexagonal protege esas reglas detrás de puertos estables hace explícitas y verificables en compilación los límites entre negocio e infraestructura

PL/SQL para la transición de estado: es la operación más critica en concurrencia; un procedimiento permite usar FOR UPDATE NOWAIT y resolver: verificar versión, validar, insertar historico, actualizar en una sola unidad, sin múltiples llamados de red adicionales.

Particionamiento mensual: a 1M ordenes/mes, particionar ORDEN y ORDEN_HISTORICO mantiene cada partición en un tamaño manejable y permite archivar/purgar con DROP PARTITION

Agregaría una réplica de lectura o caché (Redis) para las consultas de listado/búsqueda y así liberar la BD primaria para escrituras, también escalado horizontal del backend detrás de un balanceador de carga/

3. Estrategia de manejo de transacciones

El limite transaccional vive en la capa de aplicación, nunca en el controlador ni en un adaptador individual. CrearOrdenService inserta la orden y su primer historico de forma atomica via JPA. ActualizarEstadoOrdenService participa, en la misma transaccion JDBC, en la llamada al procedimiento PL/SQL (el commit/rollback es siempre responsabilidad del backend, el procedimiento nunca hace commit). Las lecturas usan @Transactional(readOnly).

4. Manejo de concurrencia

Uso dos mecanismos distintos para dos problemas distintos: bloqueo optimista con @Version para detectar conflictos sin bloquear lectores, y bloqueo pesimista puntual (FOR UPDATE NOWAIT) solo dentro de la transacción critica en PL/SQL, para fallar rápido en vez de encolar esperas. Las transacciones las mantengo cortas y traduzco los errores de concurrencia de Oracle (ORA-2000x) a un 409 claro en vez de un 500.

5. Estrategia de indexacion en Oracle

ORDEN y ORDEN_HISTORICO particionadas por rango mensual (fecha_creacion / fecha_cambio), con indices globales sobre (estado, fecha_creacion), (fecha_creacion DESC), (cliente_id) y (orden_id, fecha_cambio DESC), porque los filtros del listado cruzan particiones.

6. Estrategia de versionamiento de API

Versionamiento por URI (/api/v1/orden), en el paquete infrastructure.rest.v1. Una v2 incompatible viviria en infrastructure.rest.v2 con sus propios DTOs, reutilizando los mismos casos de uso de application; el versionamiento nunca se propaga a application ni a domain.

7. Estrategia de logging y auditoria

Separo logging técnico (Logback + logstash-logback-encoder, JSON estructurado, CorrelationIdFilter con X-Correlation-Id en el MDC, rotación por días/semanas) de la auditoria de negocio, cuya fuente de verdad es ORDEN_HISTORICO (quien cambio que estado, cuando y por qué), consultable por la API y con retención de 24 meses.

8. Preguntas técnicas y del cargo

Como optimizarías una consulta que tarda 12 segundos con 10 millones de registros?

Primero trataría de revisar el plan de ejecución para ver si hace full table scan o si las estadísticas están desactualizadas. Luego reviso si el filtro permite usar índice y si estoy trayendo más de lo necesario (SELECT *, falta de paginación, N+1 del ORM). Si el problema es agregación para reportes, uso una vista materializada en vez de recalculer en cada request.

Que tipo de índices usarías?

B-Tree para columnas de alta cardinalidad en igualdad o rango (cliente_id, fechas). Compuestos cuando el filtro combina columnas juntas seguido, por ejemplo (estado, fecha_creacion). Funcionales para búsquedas case-insensitive como UPPER(nombre). Bitmap solo lo considero en reporting, no en un sistema transaccional como este. Y trato de no indexar de más, porque cada índice extra puede ralentizar cada INSERT/UPDATE.

Particionarias la tabla? ¿Como?

Si, en una tabla que crece sin parar como ORDEN_HISTORICO la particiono por rango de fecha (mensual). Así las consultas de 'últimos N días' tocan una o dos particiones, y purgar datos viejos es un DROP PARTITION en vez de un DELETE masivo. La tabla ORDEN activa la particiono igual por fecha de creación.

Como manejarías concurrencia y transacciones?

Uso dos mecanismos distintos para dos problemas distintos: bloqueo optimista con @Version para detectar conflictos sin bloquear lectores, y bloqueo pesimista puntual (FOR UPDATE NOWAIT) solo dentro de la transacción critica en PL/SQL, para fallar rápido en vez de encolar esperas. Las transacciones las mantengo cortas y traduzco los errores de concurrencia de Oracle (ORA-2000x) a un 409 claro en vez de un 500.

Como aseguras calidad técnica en el equipo?

Code review obligatorio antes de mergear, tests automatizados como gate de CI y no como sugerencia, validadores de código, pipelines que revisen esto, y una arquitectura que haga difícil cometer el error (hexagonal: el dominio no puede depender de infraestructura por diseño, no por buena voluntad). También agregaría una capa extra de IA para revisar el código antes de subirlo a producción.

Como manejas deuda técnica?

La hago visible en un backlog técnico explicito, la priorizo por riesgo real y se va realizando de forma incremental junto con features relacionadas, en vez de pedir sprints de refactor aislados que casi nunca se aprueban. Usaría IA para apoyarme en solucionar esta deuda técnica mucho más rápido y eficiente.

Como tomas decisiones cuando el equipo no esta de acuerdo?

Bajo la discusión de opinión a criterios: que problema resuelve cada opción, que tan reversible es, y si hay una forma barata de probarlo con datos. Si sigue empatado decido yo como responsable y explico el porqué, dejando la puerta abierta a cambiar si la evidencia lo pide.

Como manejas un desarrollador de bajo desempeño?

Hablo con la persona temprano y de forma directa, sin esperar la evaluación formal. Trato de identificar si es un tema de habilidad, de expectativas poco claras, o algo personal, porque cada uno se resuelve distinto, y doy seguimiento con plazos concretos.

Como defines estándares de desarrollo?

Los defino junto con el equipo, no de forma unilateral, y los hago cumplir con herramientas (linters, formatters, plantillas de PR, CI que bloquea el merge) en vez de depender de que alguien se acuerde.

Como asegurarías calidad en entregas criticas?

Pruebas automatizadas en lo que más importa: la lógica de negocio y los puntos de integración riesgosos, como las transiciones de estado via PL/SQL en este proyecto. Además, despliegues progresivos para reducir el impacto de un error y un plan de rollback claro antes de salir a producción, no improvisado después de que algo se rompe.